

1. How to implement singleton in Java → Lazy Singleton

For Lazy singleton I use Double-Checked Locking. Declare the instance as private static volatile - volatile prevents instruction reordering so no thread partially initialize object. In `getInstance()`, first null check skips the lock for performance once instance exists, synchronized ensures only 1 thread creates it, second null check inside handles the case where 2 threads pass the check simultaneously. Alternatively the static inner class approach is cleaner - holder class only loads on first call, thread safe by JVM.

2. New Java features in Java 8 / Java 11 / Java 17 / Java 21 ?

① Java 8: Lambda, Stream API, Optional (a container may or may not hold value), Functional interface, Default method

② Java 11: var keyword (NOT Recommended), HTTP Client.

③ Java 17: Records - a concise IMMUTABLE data class - We just declare the fields and Java auto-generates constructor, getters, equals, hashCode, and toString.

④ Sealed classes, Text Blocks, Pattern matching for `InstanceOf`

⑤ Java 21: Virtual threads - Lightweight threads managed by JVM
Sequenced Collections, Record Patterns, Pattern matching for switch.

3. prevent memory leak ?

Memory leaks happens objects are no longer needed but still Referenced, so GC can't reclaim them. Common causes: static fields holding large objects since they live in method area and never got collected, ~~the~~ unclosed resources like DB connections - always use try-with-resources, listeners that are never deregistered. And `ThreadLocal` variables in thread pools are never removed after use.

To solve this, in production I add `-XX:+HeapDumpOnOutOfMemoryError` to automatically capture a heap dump on OOM, then analyze it with Eclipse MAT to find the leak.

4. What is String constant pool for?

String constant pool is a special area in the Heap, where the JVM stores String literals to save memory. Eg. when I write `String a = "hello"`, the JVM checks if "hello" already exists in the pool - if it does, it reuses the same reference instead of creating a new object. So two variables with the same string literal actually point to the same object in memory. This is why `==` can return true for 2 String literals even though you didn't create them explicitly as the same object.

However if I use `new String("hello")`, it always creates a new object in the Heap outside the pool. That's why I always use `equals()` to compare String instead of `==`.

And String is immutable - since one can modify a String after creation, it's safe to share the same instance across multiple references without any thread safety concerns.

5. Interface ~~vs~~ vs abstract class

Both are used to ~~achieve~~ achieve abstraction but serve diff purpose. Key difference is a class can implement multiple interfaces but only extend one abstract class.

Abstract class can have fields, constructors and concrete methods - interface can't have fields or constructors, only abstract methods before Java 8, but since Java 8 it can have default and static methods. I use interfaces to define ~~or~~ ~~contracts~~ contracts for unrelated classes like Runnable or my service layer interface. I use abstract classes when there's shared state or common behavior among related classes.

6. How can you avoid race condition

Race conditions happen when multiple threads access shared mutable state without synchronization. There are several ways to avoid them.

Synchronized blocks for mutual exclusion - only one thread can enter at a time but always minimized the critical section, don't lock the entire method.

AtomicInteger and atomic classes use ~~Atomic~~ ~~AtomicInteger~~ CAS - Compare And Swap - under the hood. Before writing, it checks whether the current value is still what it expects. If yes, it writes the new value. If not, it means another thread already changed it, so it retries. This is lock-free and more performant than synchronized when ~~not~~ contention is low.

Volatile for simple flags to guarantee visibilities and prevent instruction ~~reordering~~ reordering, but it doesn't guarantee atomicity so not suitable for count++.

ConcurrentHashMap and CopyOnWriteArrayList for thread-safe collections

And immutability - the best way to avoid race condition is to not have mutable shared state at all. immutable objects are thread safe by design.

In practice in web applications most concurrency is handled at the database layer ~~to~~ through transactions, not in the service layer.